Foundations of Spatial AI

Lecture 10 — Differentiable Primitive Rendering

Detailed lecture notes

Based on the slides by Emanuele Giacomini and Luca Di Giammarino
Sapienza University of Rome

October 26, 2026

Contents

1 Differentiable Primitive Rendering *(slides 1–7)* **2**

1.1 Recap: Rendering Volumes *(slide 3)* . . . 2
1.2 Rendering Primitives *(slides 4–6)* . . . 2
1.3 Today's Goals *(slide 7)* . . . 3

2 Mesh Rendering via Ray-casting *(slides 8–13)* **3**

2.1 Mesh Rendering via Ray-casting *(slide 8)* . . . 3
2.2 Mesh Rendering: Ray-Triangle Intersection *(slide 9)* . . . 3
2.3 Mesh Rendering: Ray-Mesh Intersection *(slide 10)* . . . 4
2.4 Mesh Rendering *(slide 11)* . . . 5
2.5 Mesh Rendering: Appearance *(slide 12)* . . . 5
2.6 Mesh Rendering (differentiable?) *(slide 13)* . . . 5

3 Mesh Rendering: Differentiability *(slide 14)* **5**

4 Differentiable Mesh Rendering *(slides 15–20)* **6**

4.1 Make triangles 'soft' *(slides 15–16)* . . . 6
4.2 Aggregate from multiple triangles *(slide 17)* . . . 7
4.3 Effect of γ and σ, and an example *(slides 18–19)* . . . 8
4.4 Differentiable Mesh Rendering via Ray tracing *(slide 20)* . . . 8

5 From Ray tracing to Rasterization *(slides 21–25)* **9**

5.1 Redundant computation *(slide 21)* . . . 9
5.2 From Ray tracing to Rasterization *(slides 22–23)* . . . 9
5.3 Differentiable Primitive Rendering *(slides 24–25)* . . . 10

6 Differentiable Gaussian Rendering *(slides 26–38)* **11**

6.1 What is the representation of a 3D gaussian? *(slides 26–29)* . . . 11
6.1.1 Gaussian primitive *(slide 27)* . . . 11
6.1.2 What Each Gaussian Stores *(slide 28)* . . . 12
6.1.3 Why Not Optimize the Covariance Directly? *(slide 29)* . . . 12
6.2 How to project to 2D and rasterize? *(slides 30–34)* . . . 12
6.2.1 World to camera *(slides 30–31)* . . . 12
6.2.2 Projection to the image *(slides 32–33)* . . . 13
6.2.3 Rasterize *(slide 34)* . . . 14
6.3 Tile Rasterizer *(slides 35–37)* . . . 14
6.3.1 Tile Rasterizer 1/3: one instance per touched tile *(slide 35)* . . . 14
6.3.2 Tile Rasterizer 2/3: one radix sort *(slide 36)* . . . 15
6.3.3 Tile Rasterizer 3/3: Blend Each Tile *(slide 37)* . . . 15
6.4 How to model/aggregate appearance? *(slide 38)* . . . 16

7 3D Gaussian Splatting: The Full Pipeline *(slides 39–41)* **17**
7.1 The Full Pipeline *(slide 39)* . . . 17
7.2 Backward Pass: Gradients Through Blending *(slide 40)* . . . 17
7.3 Optimization *(slide 41)* . . . 18

8 Initialization, Densification and Ablations *(slides 42–45)* **18**
8.1 Initialization: SfM Points vs Random *(slide 42)* . . . 18
8.2 Adaptive Density Control *(slide 43)* . . . 18
8.3 Ablation: Densification *(slide 44)* . . . 20
8.4 Ablation: Anisotropy *(slide 45)* . . . 20

9 Results: Quality and Speed *(slides 46–48)* **21**

10 Limitations *(slide 49)* **21**

11 Other Primitives *(slides 50–54)* **22**
11.1 Other Primitives: Fuzzy Metaballs *(slide 50)* . . . 22
11.2 2D Gaussian Splatting: The Primitive *(slide 51)* . . . 22
11.3 2D Gaussian Splatting: Ray-Splat Intersection *(slide 52)* . . . 23
11.4 From Cameras to LiDAR: Splat-LOAM *(slide 53)* . . . 24
11.5 Beyond Gaussians: Other Primitives *(slide 54)* . . . 24

12 Summary *(slide 55)* **25**

How to read these notes. Section and subsection titles follow the slide titles, and each carries its slide range. No companion book chapter was supplied, so the formal material (definitions, lemmas, proofs, derivations) was added from the standard sources the slides cite: Ravi et al. (PyTorch3D), Liu et al. (Soft Rasterizer), Zwicker et al. (EWA splatting), Kerbl et al. (3D Gaussian Splatting) and Huang et al. (2D Gaussian Splatting). Statements that go beyond the slides are labelled as such. Dashed boxes mark photos or screenshots to be copied from the indicated slide.

Notation. The slides reuse a few symbols in different sections; the context fixes the meaning.

Symbol	Meaning
$\boldsymbol{x}_0,\ \boldsymbol{\omega},\ \lambda$	ray origin, unit direction, distance along the ray
$\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3$	triangle vertices; (α, β) barycentric coordinates (mesh sections)
$\boldsymbol{c},\ \boldsymbol{c}_k$	RGB colour of a pixel / of primitive k
$m,\ \sigma,\ \gamma$	soft mask, mask sharpness, depth-weighting temperature (soft meshes)
$\boldsymbol{\mu},\ \boldsymbol{p},\ \boldsymbol{\Sigma}$	mean (position) and covariance of a Gaussian
$\boldsymbol{R},\ \boldsymbol{S}$	rotation and diagonal scale matrix, $\boldsymbol{\Sigma} = \boldsymbol{R}\boldsymbol{S}\boldsymbol{S}^\top\boldsymbol{R}^\top$
$\alpha,\ \bar{\alpha}_k$	opacity of a Gaussian / its opacity at a given pixel (Gaussian sections)
$T_k,\ w_k$	transmittance in front of primitive k, blending weight
$\boldsymbol{W},\ \boldsymbol{t},\ \boldsymbol{K}$	world→camera rotation, translation, camera intrinsics
$\pi,\ \boldsymbol{J}$	projection function, its Jacobian at the Gaussian mean

1 Differentiable Primitive Rendering *(slides 1–7)*

The question of this lecture is how to render a scene made of *primitives* (triangles, Gaussians, voxels, ...) so that the image is a differentiable function of the primitive parameters. With such a renderer we can recover the scene by gradient descent on an image loss.

1.1 Recap: Rendering Volumes *(slide 3)*

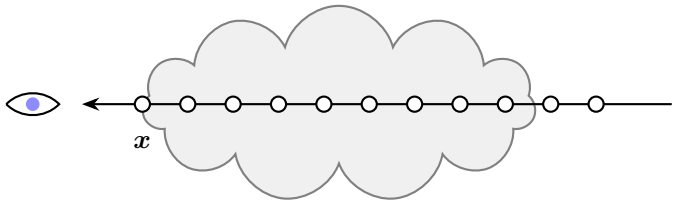
Volume rendering has two steps: (1) draw samples along the ray, (2) aggregate their contributions.

Definition 1.1 (Discrete volume rendering)**.** Let a ray $\boldsymbol{r}(t) = \boldsymbol{x}_0 + t\boldsymbol{\omega}$ be sampled at $t_1 < \cdots < t_N$ with spacings δ_i, densities σ_i and colours $\boldsymbol{c}_i$. Then

$$\boldsymbol{c} = \sum_{i=1}^{N} T_i \, \alpha_i \, \boldsymbol{c}_i, \qquad \alpha_i = 1 - e^{-\sigma_i \delta_i}, \qquad T_i = \prod_{j<i} (1 - \alpha_j).$$

Intuition

T_i is the fraction of light that survives up to sample i, and α_i is the share that sample i absorbs and returns as its own colour. Everything in this lecture is a variation of this "front-to-back" bookkeeping. The only thing that changes is *where* the contributions come from.



1. draw samples along the ray 2. aggregate their contributions

Figure 1: Volume rendering: samples are drawn along the ray through a volume and then aggregated.

1.2 Rendering Primitives *(slides 4–6)*

Gaussians (slide 4). The same two steps work for primitives such as Gaussians: sample along the ray, aggregate whatever the samples hit.

Triangles/meshes (slide 5). Here step 1 is *wasteful*: we know exactly where the primitives are, so most samples fall in empty space.

The primitive view (slide 6). Replace sampling by a direct query:

1. **Find the primitives that affect the ray/pixel;**
2. **aggregate their contributions** to render the pixel.

Definition 1.2 (Primitive-based rendering)**.** A scene is a set of primitives $\{\mathsf{p}_k\}$, each with a geometry and an appearance. For a ray $\boldsymbol{r}$ let $\mathcal{P}(\boldsymbol{r}) = \{k : \mathsf{p}_k \text{ affects } \boldsymbol{r}\}$ be the *influence set*. The pixel value is $\boldsymbol{c}(\boldsymbol{r}) = \mathrm{Agg}\big(\{(\lambda_k, \boldsymbol{c}_k, \dots)\}_{k \in \mathcal{P}(\boldsymbol{r})}\big)$, where λ_k is the depth of primitive k along the ray.

Intuition

The two design questions of the lecture are: how to compute $\mathcal{P}(\boldsymbol{r})$ *efficiently* (ray casting, then rasterization), and how to choose Agg so that the result is *differentiable*.

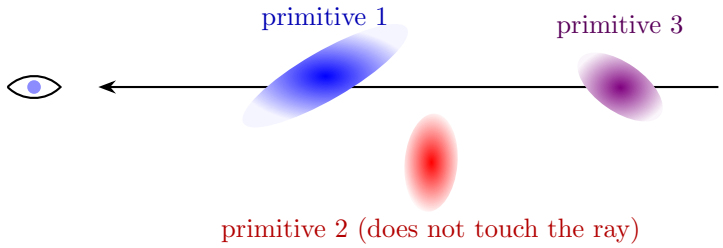


Figure 2: Only primitives 1 and 3 are in the influence set $\mathcal{P}(\boldsymbol{r})$ of this ray; primitive 2 is not.

1.3 Today's Goals *(slide 7)*

1. (Differentiable) mesh rendering via ray tracing.
2. From ray tracing to rasterization.
3. 3D Gaussian Splatting in detail: projection, tile rasterizer, optimization.

2 Mesh Rendering via Ray-casting *(slides 8–13)*

2.1 Mesh Rendering via Ray-casting *(slide 8)*

For each pixel we cast a ray and (1) identify the *closest* intersecting triangle, then (2) compute the appearance at the intersection point. Note that a *mesh* is equivalent to a *triangle set* for this algorithm: connectivity plays no role.

[**insert image here**]
taken from **slide 8**

Figure 3: Ray cast from a pixel of the image plane through the cow mesh, hitting the closest triangle (red dot on the image plane, red arrow reaching the surface).

2.2 Mesh Rendering: Ray-Triangle Intersection *(slide 9)*

Definition 2.1 (Barycentric coordinates)**.** Given a triangle $(\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3)$, every point of its plane is written $\boldsymbol{p} = \alpha\boldsymbol{v}_1 + \beta\boldsymbol{v}_2 + (1-\alpha-\beta)\boldsymbol{v}_3$. The pair (α, β) are the barycentric coordinates of $\boldsymbol{p}$. The point lies inside the triangle iff $\alpha, \beta, 1-\alpha-\beta \in (0,1)$.

For a ray $(\boldsymbol{x}_0, \boldsymbol{\omega})$ we need the point that is both on the ray and in the triangle plane:

$$\boldsymbol{p} = \boldsymbol{x}_0 + \lambda\boldsymbol{\omega} = \alpha\boldsymbol{v}_1 + \beta\boldsymbol{v}_2 + (1-\alpha-\beta)\boldsymbol{v}_3.$$

This is a linear system in (α, β, λ).

Proposition 2.2 (Closed-form ray–triangle intersection)**.** *Let* $\boldsymbol{e}_1 = \boldsymbol{v}_1 - \boldsymbol{v}_3$, $\boldsymbol{e}_2 = \boldsymbol{v}_2 - \boldsymbol{v}_3$, $\boldsymbol{s} = \boldsymbol{x}_0 - \boldsymbol{v}_3$, *and* $\boldsymbol{h} = \boldsymbol{\omega} \times \boldsymbol{e}_2$, $a = \boldsymbol{e}_1 \cdot \boldsymbol{h}$, $\boldsymbol{q} = \boldsymbol{s} \times \boldsymbol{e}_1$. *If* $a \neq 0$ *(the ray is not parallel to the triangle),*

$$\alpha = \frac{\boldsymbol{s} \cdot \boldsymbol{h}}{a}, \qquad \beta = \frac{\boldsymbol{\omega} \cdot \boldsymbol{q}}{a}, \qquad \lambda = \frac{\boldsymbol{e}_2 \cdot \boldsymbol{q}}{a}.$$

Proof. Subtracting $\boldsymbol{v}_3$ from both sides gives $\alpha\boldsymbol{e}_1 + \beta\boldsymbol{e}_2 - \lambda\boldsymbol{\omega} = \boldsymbol{s}$, i.e. $[\boldsymbol{e}_1\ \boldsymbol{e}_2\ -\boldsymbol{\omega}]\,(\alpha, \beta, \lambda)^{\mathsf{T}} = \boldsymbol{s}$. By Cramer's rule with $D = \det[\boldsymbol{e}_1\ \boldsymbol{e}_2\ -\boldsymbol{\omega}] = -\boldsymbol{\omega} \cdot (\boldsymbol{e}_1 \times \boldsymbol{e}_2)$, and the cyclic invariance of the scalar triple product, one finds $D = -a$ and $\alpha = \det[\boldsymbol{s}\ \boldsymbol{e}_2\ -\boldsymbol{\omega}]/D = \boldsymbol{s} \cdot \boldsymbol{h}/a$, $\beta = \det[\boldsymbol{e}_1\ \boldsymbol{s}\ -\boldsymbol{\omega}]/D = \boldsymbol{\omega} \cdot \boldsymbol{q}/a$, $\lambda = \det[\boldsymbol{e}_1\ \boldsymbol{e}_2\ \boldsymbol{s}]/D = \boldsymbol{e}_2 \cdot \boldsymbol{q}/a$. □

Intuition

Think of two questions answered at once: "where does the ray pierce the plane of the triangle?" (that is $\lambda = -\boldsymbol{s} \cdot \boldsymbol{n}/\boldsymbol{\omega} \cdot \boldsymbol{n}$ with $\boldsymbol{n} = \boldsymbol{e}_1 \times \boldsymbol{e}_2$) and "where is that point expressed in the triangle's own (α, β) frame?". Every ingredient is a smooth function of the vertices, which is why (α, β, λ) will turn out to be differentiable w.r.t. vertex positions.

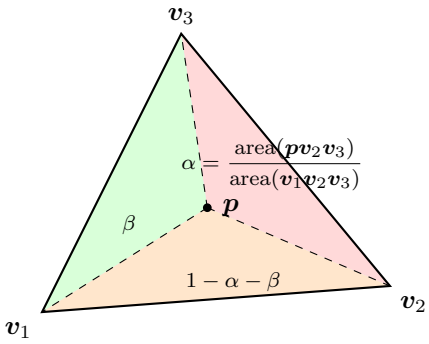


Figure 4: Barycentric coordinates as area ratios: the weight of a vertex is the relative area of the sub-triangle *opposite* to it (β is the area of $\boldsymbol{p}\boldsymbol{v}_1\boldsymbol{v}_3$, $1 - \alpha - \beta$ the area of $\boldsymbol{p}\boldsymbol{v}_1\boldsymbol{v}_2$, both over the total).

2.3 Mesh Rendering: Ray-Mesh Intersection *(slide 10)*

Algorithm 2.3 (Closest intersecting triangle)**.**

```
def mesh_ray_intersection(mesh, ray):
lam_min = inf
intersection = None
for triangle in mesh:
    (alpha, beta, lam) = intersect(ray, triangle)
    # inside test: 0 < alpha, beta, (1-alpha-beta) < 1
    if inside(alpha, beta) and 0 < lam < lam_min:
        lam_min = lam
        intersection = (triangle, (alpha, beta))
return intersection
```

Remark 2.4. The slide's code is explicitly "rough pseudo-code". It does not update `lam_min`, which a working version must do, otherwise the *closest* hit is not guaranteed. The cost is one intersection test per triangle *per ray*.

2.4 Mesh Rendering *(slide 11)*

The two steps are two maps:

$$(1)\ \ \text{mesh}, \text{ray} \ \to\ \text{triangle}, (\alpha, \beta) \qquad (2)\ \ \text{triangle}, (\alpha, \beta) \ \to\ \boldsymbol{c} \in \mathbb{R}^3.$$

2.5 Mesh Rendering: Appearance *(slide 12)*

Definition 2.5 (Per-vertex colour and texture atlas)**. (a) Per-vertex colour:** each vertex carries a colour, $(\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3) \to (\boldsymbol{c}_1, \boldsymbol{c}_2, \boldsymbol{c}_3)$, and $\boldsymbol{c} = \alpha\boldsymbol{c}_1 + \beta\boldsymbol{c}_2 + (1-\alpha-\beta)\boldsymbol{c}_3$. **(b) Texture atlas:** each triangle owns a $C \times C$ texture image and $\boldsymbol{c} = \text{bilinear_sample}(\text{atlas}, [\alpha, \beta])$.

Appearance can be extended to lighting-dependent shading, using the albedo as stored texture together with the surface normals.

Example 2.6. Let $\boldsymbol{c}_1 = (1, 0, 0)$, $\boldsymbol{c}_2 = (0, 1, 0)$, $\boldsymbol{c}_3 = (0, 0, 1)$ and $(\alpha, \beta) = (0.2, 0.5)$. Then $\boldsymbol{c} = 0.2\,\boldsymbol{c}_1 + 0.5\,\boldsymbol{c}_2 + 0.3\,\boldsymbol{c}_3 = (0.2, 0.5, 0.3)$.

Intuition

Per-vertex colour is a smooth "gradient" blended over the triangle (cheap, low resolution). A texture atlas spends memory to store detail inside each face. In both cases the colour is a *linear/bilinear* function of (α, β), hence trivially differentiable w.r.t. the barycentric coordinates.

2.6 Mesh Rendering (differentiable?) *(slide 13)*

Step 1 gives (α, β) that are differentiable w.r.t. the mesh vertex positions (Proposition 2.2); step 2 gives a colour differentiable w.r.t. the barycentric coordinates. The slide asks "Is this differentiable?" and answers: *well, sort of...* The next section shows exactly where it fails.

3 Mesh Rendering: Differentiability *(slide 14)*

The rendered colour *is* differentiable w.r.t. the vertex positions of the visible triangle. For example, we can make the colour more red by changing the position of the triangle (it changes which part of the colour gradient lands on the pixel). Two things are not differentiable:

1. rendering is discontinuous w.r.t. **occlusion/dis-occlusion**;
2. rendering is discontinuous w.r.t. **depth ordering**.

Proposition 3.1 (Hard rendering is piecewise smooth)**.** *Let $k^\star(\theta) = \arg\min_{k\,:\,\text{ray hits } k} \lambda_k(\theta)$ be the visible triangle for scene parameters θ. The pixel value is $\boldsymbol{c}(\theta) = \boldsymbol{c}_{k^\star(\theta)}(\theta)$. On every region of parameter space where $k^\star$ is constant, $\boldsymbol{c}$ is smooth. Across a boundary where the visible triangle changes, $\boldsymbol{c}$ jumps. Consequently the gradient is either undefined at the jump or carries no information about it.*

Intuition

Imagine you are blindfolded and can only feel the slope of the image at your feet. If the pixel is currently yellow, the slope of "colour vs. z" is flat: nothing you can feel says "pushing the yellow face back would make the pixel red". Likewise an *occluded* triangle has *exactly zero*

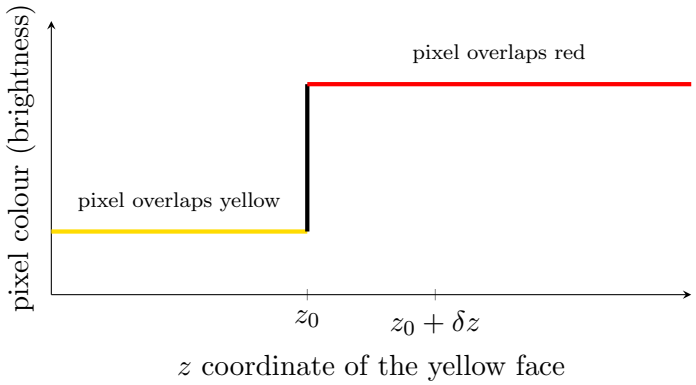


Figure 5: Depth-ordering discontinuity (redrawn from the diagram on slide 14): as the yellow face is shifted behind the red one, the pixel colour jumps from yellow to red at a single z. The derivative is 0 almost everywhere and undefined at the jump.

[**insert image here**]
taken from **slide 14**

Figure 6: Original 3D sketch: shifting the yellow triangle by $+\delta z$ changes which face the pixel overlaps (left), and the occlusion example with an orange and a purple triangle (right).

influence on the image, so it never receives a gradient telling it to move into view.

4 Differentiable Mesh Rendering *(slides 15–20)*

The fix (Ravi et al., PyTorch3D, 2020; Liu et al., Soft Rasterizer, ICCV 2019) has two parts: (1) make triangles *soft*, (2) aggregate from *multiple* triangles.

4.1 Make triangles 'soft' *(slides 15–16)*

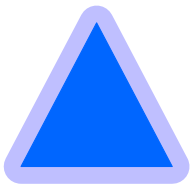
Definition 4.1 (Soft triangle mask)**.** Let Δ be the signed distance, in pixel space, from the pixel to the triangle ($\Delta < 0$ inside, > 0 outside). The soft mask is

$$m = \mathrm{sigmoid}\Big(\frac{-\Delta}{\sigma}\Big) = \frac{1}{1 + e^{\Delta/\sigma}},$$

with a manually defined cut-off (e.g. 10σ) beyond which the mask is set to zero.

Proposition 4.2 (Properties of the soft mask)**.** *(i) $m \to 1$ deep inside and $m \to 0$ far outside; $m = \frac{1}{2}$ on the boundary. (ii) $\partial m/\partial \Delta = -m(1-m)/\sigma$ is non-zero for every pixel near the triangle. (iii) As $\sigma \to 0$, m converges to the hard indicator of the triangle.*

Proof. (i)–(iii) follow from the shape of the logistic function; (ii) is its derivative $\sigma'(x) = \sigma(x)(1 -$



soft triangle
(fuzzy border of width r)

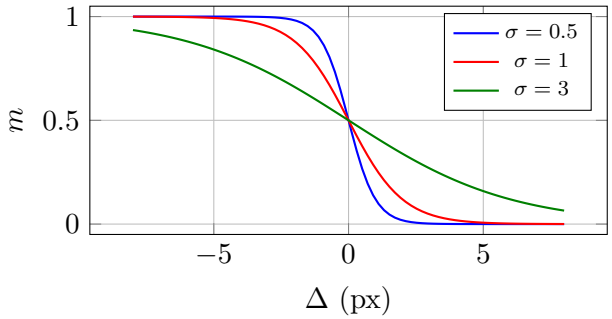


Figure 7: Left: a soft triangle has a fuzzy border of width r (slide 15–16). Right: the mask $m = \text{sigmoid}(-\Delta/\sigma)$ for several σ; $m = 0.5$ exactly on the boundary.

$\sigma(x)$) with the chain rule on $x = -\Delta/\sigma$. □

Example 4.3. With $\sigma = 1$: a pixel 2 px *outside* ($\Delta = 2$) gets $m = \text{sigmoid}(-2) = 0.119$, while one 3 px *inside* ($\Delta = -3$) gets $m = \text{sigmoid}(3) = 0.953$. With $\sigma = 2$ the outside pixel gets $\text{sigmoid}(-1) = 0.269$: a larger σ blurs the silhouette.

Intuition

A hard triangle is a cookie cutter: a pixel is either in or out. A soft triangle is a cookie with a fuzzy rim. A pixel just outside the rim still "feels" the triangle a little, and therefore the loss on that pixel can pull the triangle's vertices towards it. This is the gradient signal that hard rasterization lacks.

4.2 Aggregate from multiple triangles *(slide 17)*

For each pixel:

1. find the K closest triangles that have a non-zero mask at the pixel (same code as for the single closest triangle);
2. weight each triangle by its mask and distance along the ray, $w_k \propto m_k\, e^{-\lambda_k/\gamma}$ (a small default weight w_{bg} can model the background);
3. combine the per-triangle appearance, $\boldsymbol{c} = \sum_k w_k \boldsymbol{c}_k$.

Definition 4.4 (Soft blending of K triangles)**.** With the weights normalised to sum to one,

$$w_k = \frac{m_k\, e^{-\lambda_k/\gamma}}{\sum_{j=1}^{K} m_j\, e^{-\lambda_j/\gamma} + w_{bg}}, \qquad \boldsymbol{c} = \sum_{k=1}^{K} w_k \boldsymbol{c}_k + w'_{bg}\, \boldsymbol{c}_{bg},$$

where $w'_{bg} = w_{bg}/(\sum_j m_j e^{-\lambda_j/\gamma} + w_{bg})$ is the background's share.

Proposition 4.5 (Limits and gradient of soft blending)**.** *(a) If $m_j > 0$ and $\lambda_j < \lambda_k$ then $w_k/w_j = (m_k/m_j)\, e^{-(\lambda_k - \lambda_j)/\gamma} \to 0$ as $\gamma \to 0$: the closest triangle wins and we recover the hard z-buffer. (b) For fixed masks and no background, $\dfrac{\partial \boldsymbol{c}}{\partial \lambda_k} = -\dfrac{w_k}{\gamma}\,(\boldsymbol{c}_k - \boldsymbol{c})$.*

Proof. (a) is the ratio of two weights. (b) The weights are a softmax of $-\lambda_j/\gamma + \log m_j$, so $\partial w_j/\partial \lambda_k = -\frac{1}{\gamma} w_j(\delta_{jk} - w_k)$. Hence $\partial \boldsymbol{c}/\partial \lambda_k = \sum_j \boldsymbol{c}_j\, \partial w_j/\partial \lambda_k = -\frac{1}{\gamma}(w_k \boldsymbol{c}_k - w_k \sum_j w_j \boldsymbol{c}_j)$. □

Example 4.6. Two fully visible triangles ($m = 1$) at depths $\lambda_1 = 1$, $\lambda_2 = 2$. For $\gamma = 1$ the weights are $\propto e^{-1} = 0.368$ and $e^{-2} = 0.135$, i.e. $w = (0.731, 0.269)$. For $\gamma = 0.1$ the ratio is $e^{-10} \approx 4.5 \cdot 10^{-5}$, so $w \approx (0.99995, 0.00005)$: almost a hard z-buffer.

Intuition

The mask answers "is this triangle over the pixel?", the exponential answers "how much does this one deserve to be seen, given how far it is?". γ is a temperature: large γ lets triangles behind show through (semi-transparency, good gradients), small γ is a crisp opaque rendering. By Proposition 4.5(b) even a partially hidden triangle ($w_k > 0$) gets a gradient that nudges it towards or away from the camera, which fixes the depth-ordering problem of the previous section.

4.3 Effect of γ and σ, and an example *(slides 18–19)*

[**insert image here**]
taken from **slide 18**

Figure 8: Cube rendered with standard rendering (left), with a larger γ (middle: back faces show through) and with larger γ and σ (right: blurred silhouette). From Liu et al., ICCV 2019.

[**insert image here**]
taken from **slide 19**

Figure 9: Soft Rasterizer example: a mesh is optimised from an initialization towards a target image/silhouette (the red hand is the part that has to move); the right part shows intermediate optimisation results.

4.4 Differentiable Mesh Rendering via Ray tracing *(slide 20)*

1. Identify the closest K 'softly' intersecting triangles (and the corresponding pixel-space SDF);
2. aggregate appearance using masks, depth-dependent weights, and per-triangle appearance.

Algorithm 4.7 (Ray-traced soft mesh rendering, slide 21 structure)**.**

```
def render(mesh, camera):
### Iterate over all pixels
for pixel in camera.grid:
    ray = pixel2ray(camera, pixel)
```

```
        ### Iterate over all triangles
        for triangle in mesh:
            # Find current K closest intersecting triangles
        ### Iterate over triangles influencing this pixel
        for triangle in sorted_k_closest:
            # Aggregate appearance
```

5 From Ray tracing to Rasterization *(slides 21–25)*

5.1 Redundant computation *(slide 21)*

In the code above, neighbouring pixels (two rays on the cow's nose) loop over the *same* triangles and redo much of the same work. **Idea: iterate over triangles just once, and store which pixels they affect.**

Proposition 5.1 (Cost of the two loop orders)**.** *Let P be the number of pixels, N the number of primitives, $\bar{f}$ the average number of pixels a primitive's footprint covers and K the number of triangles kept per pixel. Pixel-major (ray tracing) costs $\Theta(PN)$ intersection tests. Primitive-major (rasterization + blending) costs $\Theta(N\bar{f})$ to fill the per-pixel structure plus $\Theta(PK)$ to blend.*

Example 5.2. $P = 10^6$ pixels, $N = 10^6$ triangles, $\bar{f} = 10$, $K = 10$: pixel-major needs 10^{12} tests; primitive-major needs about $10^7 + 10^7 = 2 \cdot 10^7$ operations (ignoring the $\log K$ factor of buffer insertion), five orders of magnitude less.

Intuition

Instead of asking every pixel "who is on top of you?", we ask every triangle "whom do you cover?". A triangle is small, so it covers few pixels, and the answer is cheap. Ray tracing is a *gather* (pixels pull), rasterization is a *scatter* (primitives push).

5.2 From Ray tracing to Rasterization *(slides 22–23)*

Ray tracing

```
for pixel in camera.grid:
  for triangle in mesh:
    # keep K closest
  for triangle in sorted_k_closest:
    # aggregate
```

Rasterization + Blending

```
for triangle in mesh:
  tri_2d = project(triangle, image)
  # update K-buffer for pixels in tri_2d
```

Rasterization

```
for pixel in camera.grid:
  for triangle in k_closest_img[pixel]:
    # aggregate
```

Blending

Figure 10: Swapping the loop order: the K closest triangles per pixel are found while iterating over triangles once, and the pixel loop only blends (slide 22).

Algorithm 5.3 (Rasterization-based soft mesh rendering)**.**

```
def render(mesh, camera):
    ### some structure to store K triangles for each pixel
```

```
sorted_k_closest_img = ...
### Iterate over all triangles   (Rasterization)
for triangle in mesh:
    tri_2d = project(triangle, image)
    # Update closest_K_img for pixels within tri_2d
### Iterate over all pixels      (Blending + Shading)
for pixel in camera.grid:
    for triangle in sorted_k_closest_img[pixel]:
        # Aggregate appearance
```

On slide 23 the PyTorch3D renderer is drawn as the pipeline below.

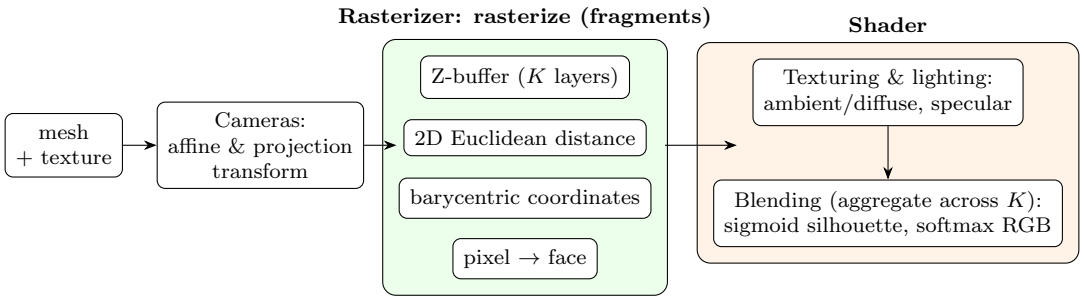


Figure 11: The PyTorch3D renderer (Ravi et al., 2020), redrawn from slide 23: a rasterizer produces per-pixel fragments (depth, distance, barycentrics, face index, K layers) and a shader textures, lights and blends them. The sigmoid silhouette is the soft mask m and the softmax RGB is the depth-weighted combination above.

5.3 Differentiable Primitive Rendering *(slides 24–25)*

The loop structure is independent of the primitive type:

Algorithm 5.4 (Generic differentiable primitive rendering)**.**

```
def render(primitives, camera):
# Initialize a rasterization data structure
# (records influencing primitives for each pixel)
for primitive in primitives:                    # Rasterization
    prim2d = project(primitive, camera)
    # Update rasterization data structure
for pixel in camera.grid:                       # Blending
    # Aggregate appearance from influencing primitives
```

For Gaussians (slide 25) the same code is instantiated with `gaussians`, projecting each one to a 2D Gaussian (`gauss2d`). To fill it in we must answer three questions: *What is the representation of a 3D Gaussian? How to project to 2D and rasterize? How to model/aggregate appearance?*

[**insert image here**]
taken from **slide 26**

Figure 12: A real scene (forest, left) and a view of the same scene where the individual Gaussians are visible as flat shaded blobs (right).

6 Differentiable Gaussian Rendering *(slides 26–38)*

6.1 What is the representation of a 3D gaussian? *(slides 26–29)*

6.1.1 Gaussian primitive *(slide 27)*

Definition 6.1 (3D Gaussian primitive)**.** A Gaussian primitive is the tuple $(\boldsymbol{p}, \boldsymbol{\Sigma}, \alpha, \boldsymbol{c})$ with density

$$\mathcal{G}_{\boldsymbol{\Sigma}}(\boldsymbol{x}-\boldsymbol{p}) = \frac{1}{(2\pi)^{3/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\Big(-\tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{p})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{p})\Big),$$

an opacity α, and a view-dependent colour $\boldsymbol{c}$ given by spherical-harmonics (SH) coefficients. The covariance is factorised into scale and rotation,

$$\boldsymbol{\Sigma} = \boldsymbol{R}\boldsymbol{S}\boldsymbol{S}^{\mathsf{T}}\boldsymbol{R}^{\mathsf{T}}, \qquad \boldsymbol{S} = \operatorname{diag}(s_1, s_2, s_3).$$

The level sets $(\boldsymbol{x}-\boldsymbol{p})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{p}) = \text{const}$ are ellipsoids whose axes are the columns of $\boldsymbol{R}$ with semi-axis lengths proportional to s_i.

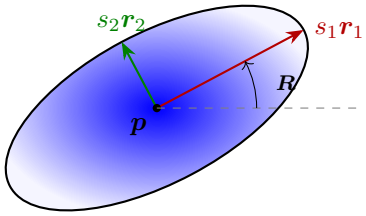


Figure 13: A 2D slice of a Gaussian: position $\boldsymbol{p}$, principal axes $\boldsymbol{r}_i$ (columns of $\boldsymbol{R}$) and scales s_i.

Remark 6.2 (Normalisation, beyond the slides). The slide writes the normalised density (with the factor $1/((2\pi)^{3/2}|\boldsymbol{\Sigma}|^{1/2})$). In the reference 3DGS implementation (Kerbl et al.) the opacity α multiplies the *unnormalised* exponential, so α is the peak opacity of the primitive. The code on slide 37 does exactly this.

Intuition

A 3D Gaussian is a soft ellipsoidal "cloud of paint": densest at the centre, fading smoothly outwards, stretched along the directions of $\boldsymbol{R}$. Because the fall-off is smooth and infinitely differentiable, small changes of $\boldsymbol{p}, \boldsymbol{s}, \boldsymbol{R}, \alpha$ change the image smoothly, which is what we need for gradients.

6.1.2 What Each Gaussian Stores *(slide 28)*

Parameter	Symbol	Floats
Position	$\boldsymbol{\mu}$	3
Scale	$\boldsymbol{s}$	3
Rotation (quaternion)	$\boldsymbol{q}$	4
Opacity	α	1
Colour, SH degree 3		48
Total		**59**

SH of degree 3 has $(3+1)^2 = 16$ coefficients per colour channel, i.e. $16 \times 3 = 48$ floats.

Example 6.3. About $3 \cdot 10^6$ Gaussians per scene, at $59 \times 4 = 236$ bytes each, give ≈ 708 MB, i.e. the "$\sim$700 MB" on the slide (consistent with the 734 MB of the results table later).

6.1.3 Why Not Optimize the Covariance Directly? *(slide 29)*

A gradient step on $\boldsymbol{\Sigma}$ can destroy its validity: the slide's example moves $\mathrm{diag}(1, 0.01, 1)$ by $-\eta\nabla\mathcal{L}$ to $\mathrm{diag}(1, -0.02, 1)$, a *negative variance*, which is not a Gaussian at all.

Proposition 6.4 (Factorised covariance is always PSD)**.** *For any rotation $\boldsymbol{R}$ and any real $\boldsymbol{S} = \mathrm{diag}(s_1, s_2, s_3)$, $\boldsymbol{\Sigma} = \boldsymbol{RSS}^\top\boldsymbol{R}^\top = (\boldsymbol{RS})(\boldsymbol{RS})^\top$ is symmetric positive semi-definite, since $\boldsymbol{x}^\top\boldsymbol{\Sigma}\boldsymbol{x} = \|\boldsymbol{S}^\top\boldsymbol{R}^\top\boldsymbol{x}\|^2 \geq 0$. Conversely, every symmetric positive definite $\boldsymbol{\Sigma}$ has such a factorisation (spectral theorem: $\boldsymbol{\Sigma} = \boldsymbol{Q\Lambda Q}^\top$, take $\boldsymbol{R} = \boldsymbol{Q}$, $\boldsymbol{S} = \boldsymbol{\Lambda}^{1/2}$).*

Intuition

Optimising $\boldsymbol{\Sigma}$ directly is like steering a car by moving the wheels' bolts around: nothing prevents an impossible configuration. Optimising $(\boldsymbol{S}, \boldsymbol{R})$ instead is steering with the wheel: whatever you do, you still get a valid ellipsoid. In the example above the update on $s_2 = 0.1$ can make s_2 any real number, but $\Sigma_{22} = s_2^2$ is never negative.

Remark 6.5 (Beyond the slides). In the reference implementation the scale is stored in log-space (activation exp) so that $s_i > 0$, the rotation is a quaternion that is normalised before building $\boldsymbol{R}$, and opacity is stored before a sigmoid so that $\alpha \in (0, 1)$. The colour is $\boldsymbol{c}(\boldsymbol{d}) = \sum_{\ell \leq 3} \sum_m k_{\ell m} Y_\ell^m(\boldsymbol{d})$ for the viewing direction $\boldsymbol{d}$.

6.2 How to project to 2D and rasterize? *(slides 30–34)*

6.2.1 World to camera *(slides 30–31)*

We use the camera extrinsics to transform each 3D Gaussian to the camera frame: $(\boldsymbol{p}, \boldsymbol{R}, \boldsymbol{S}) \to (\boldsymbol{p}', \boldsymbol{R}', \boldsymbol{S})$.

Lemma 6.6 (Affine maps keep Gaussians Gaussian)**.** *If $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ then $\boldsymbol{Ax} + \boldsymbol{b} \sim \mathcal{N}(\boldsymbol{A\mu} + \boldsymbol{b},\ \boldsymbol{A\Sigma A}^\top)$.*

Proof. The characteristic function of $\boldsymbol{x}$ is $\varphi(\boldsymbol{t}) = \exp(i\boldsymbol{t}^\top\boldsymbol{\mu} - \frac{1}{2}\boldsymbol{t}^\top\boldsymbol{\Sigma}\boldsymbol{t})$. For $\boldsymbol{y} = \boldsymbol{Ax} + \boldsymbol{b}$, $\varphi_y(\boldsymbol{t}) = e^{i\boldsymbol{t}^\top\boldsymbol{b}}\varphi(\boldsymbol{A}^\top\boldsymbol{t}) = \exp\left(i\boldsymbol{t}^\top(\boldsymbol{A\mu} + \boldsymbol{b}) - \frac{1}{2}\boldsymbol{t}^\top\boldsymbol{A\Sigma A}^\top\boldsymbol{t}\right)$, which is the characteristic function of $\mathcal{N}(\boldsymbol{A\mu} + \boldsymbol{b}, \boldsymbol{A\Sigma A}^\top)$. Note that $\boldsymbol{A}$ need not be invertible. □

World→camera is affine, so

$$\boldsymbol{\mu}_c = \boldsymbol{W}\boldsymbol{\mu} + \boldsymbol{t}, \qquad \boldsymbol{\Sigma}_c = \boldsymbol{W}\boldsymbol{\Sigma}\boldsymbol{W}^{\mathsf{T}} = (\boldsymbol{W}\boldsymbol{R})\boldsymbol{S}\boldsymbol{S}^{\mathsf{T}}(\boldsymbol{W}\boldsymbol{R})^{\mathsf{T}},$$

that is $\boldsymbol{p}' = \boldsymbol{W}\boldsymbol{p} + \boldsymbol{t}$, $\boldsymbol{R}' = \boldsymbol{W}\boldsymbol{R}$ and the scale is unchanged (slide 30).

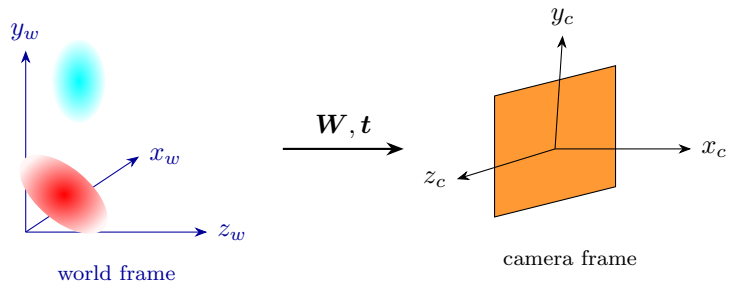


Figure 14: Gaussians are expressed in the camera frame by the affine map $(\boldsymbol{W}, \boldsymbol{t})$ (slides 30–31).

Intuition

Changing the viewpoint does not change the cloud of paint; it only changes the coordinates in which we describe it. A rigid motion rotates the ellipsoid ($\boldsymbol{R} \to \boldsymbol{W}\boldsymbol{R}$) and moves its centre, nothing else.

6.2.2 Projection to the image *(slides 32–33)*

Let π map 3D points to pixels: $\pi(\boldsymbol{x}) = \boldsymbol{u}$ with $z\,(u, v, 1)^{\mathsf{T}} = \boldsymbol{K}\,(X, Y, Z)^{\mathsf{T}}$. The 2D mean is $\boldsymbol{\mu}_{2D} = \pi(\boldsymbol{\mu}_{3D})$. *What is the image-space projection of a 3D Gaussian?* It can be approximated as a 2D Gaussian (EWA Volume Splatting, Zwicker et al., 2001), with covariance

$$\boldsymbol{\Sigma}_{2D} = \boldsymbol{J}\,\boldsymbol{\Sigma}_{3D}\,\boldsymbol{J}^{\mathsf{T}}, \qquad \boldsymbol{J} = \frac{\partial \pi}{\partial \boldsymbol{x}}(\boldsymbol{\mu}_{3D}).$$

The projection is *not* affine: $\pi(\boldsymbol{x}_c) = \big(f_x X/Z + c_x,\ f_y Y/Z + c_y\big)^{\mathsf{T}}$ with $\boldsymbol{x}_c = (X, Y, Z)^{\mathsf{T}}$.

Theorem 6.7 (EWA: local affine approximation of the projection)**.** *Let $\boldsymbol{x}_c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$ with $\boldsymbol{\Sigma}_c = \boldsymbol{W}\boldsymbol{\Sigma}\boldsymbol{W}^{\mathsf{T}}$. Linearising π at the mean, $\pi(\boldsymbol{x}_c) \approx \pi(\boldsymbol{\mu}_c) + \boldsymbol{J}(\boldsymbol{x}_c - \boldsymbol{\mu}_c)$, with*

$$\boldsymbol{J} = \frac{\partial \pi}{\partial \boldsymbol{x}_c}\bigg|_{\boldsymbol{\mu}_c} = \begin{bmatrix} f_x/Z & 0 & -f_x X/Z^2 \\ 0 & f_y/Z & -f_y Y/Z^2 \end{bmatrix}_{\boldsymbol{\mu}_c},$$

the projected Gaussian is $\mathcal{N}(\boldsymbol{\mu}_{2D}, \boldsymbol{\Sigma}_{2D})$ with $\boldsymbol{\mu}_{2D} = \pi(\boldsymbol{\mu}_c)$ and $\boldsymbol{\Sigma}_{2D} = \boldsymbol{J}\boldsymbol{W}\boldsymbol{\Sigma}\boldsymbol{W}^{\mathsf{T}}\boldsymbol{J}^{\mathsf{T}} \in \mathbb{R}^{2\times 2}$.

Proof. The linearisation is an affine map of $\boldsymbol{x}_c$ with linear part $\boldsymbol{J}$. Apply Lemma 6.6. The derivatives are those of $f_x X/Z$ and $f_y Y/Z$. □

Example 6.8. Take $f_x = f_y = 500$, $(c_x, c_y) = (320, 240)$, $\boldsymbol{\mu}_c = (0.2, -0.1, 2.0)$ and $\boldsymbol{\Sigma}_c = \mathrm{diag}(0.01, 0.01, 0.04)$. Then $\boldsymbol{\mu}_{2D} = (500 \cdot 0.1 + 320,\ 500 \cdot (-0.05) + 240) = (370, 215)$ and $\boldsymbol{J} = \begin{bmatrix} 250 & 0 & -25 \\ 0 & 250 & 12.5 \end{bmatrix}$, so

$$\boldsymbol{\Sigma}_{2D} = \begin{bmatrix} 250^2 \cdot 0.01 + 25^2 \cdot 0.04 & -25 \cdot 12.5 \cdot 0.04 \\ \cdot & 250^2 \cdot 0.01 + 12.5^2 \cdot 0.04 \end{bmatrix} = \begin{bmatrix} 650 & -12.5 \\ -12.5 & 631.25 \end{bmatrix},$$

i.e. a splat with standard deviation of about 25 px. The depth uncertainty (0.04) leaks into the image through the off-axis entries of $\boldsymbol{J}$.

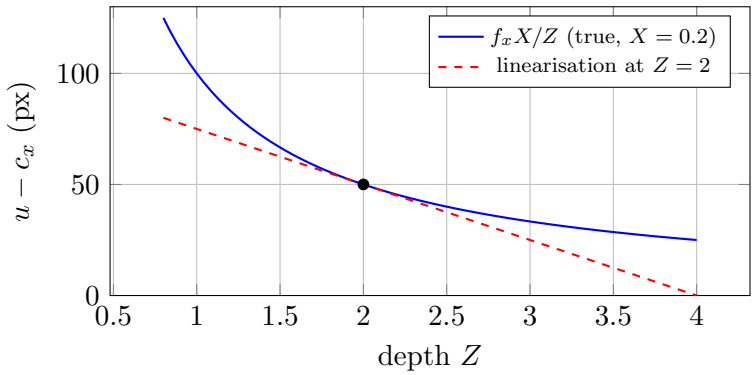


Figure 15: The linearisation is exact at the Gaussian mean and degrades away from it: the approximation is good if the Gaussian's extent is small compared with its depth.

Intuition

Seen from far away, the nonlinear perspective map looks locally like a plain linear map (a tilted ruler). If a Gaussian is small relative to its distance, replacing the true projection by its tangent line at the centre changes almost nothing, and linear maps keep Gaussians Gaussian. Large Gaussians very close to the camera are where this breaks.

6.2.3 Rasterize *(slide 34)*

1. Sort the Gaussians from closest to furthest from the camera.
2. For each pixel $\boldsymbol{u}$, compute the opacity of each Gaussian $\mathcal{G}_k$:

$$\bar{\alpha}_k = \alpha_k \frac{\exp\big(-\frac{1}{2}(\boldsymbol{u}-\boldsymbol{\mu}_{2D}^k)^\mathsf{T}(\boldsymbol{\Sigma}_{2D}^k)^{-1}(\boldsymbol{u}-\boldsymbol{\mu}_{2D}^k)\big)}{2\pi\,|\boldsymbol{\Sigma}_{2D}^k|^{0.5}}.$$

In practice one rasterizes *blocks* instead of the entire image, since not all Gaussians influence all blocks. (As in the remark above, implementations drop the normalising denominator so that $\bar{\alpha}_k \le \alpha_k$.)

6.3 Tile Rasterizer *(slides 35–37)*

6.3.1 Tile Rasterizer 1/3: one instance per touched tile *(slide 35)*

1. Cull Gaussians outside the frustum.
2. Split the image into 16×16-pixel tiles.
3. Make one copy (*instance*) of the Gaussian for every tile it touches.
4. Give each instance the key = (tile ID, view depth), a 64-bit integer: tile ID in the high 32 bits, depth in the low 32 bits.

Remark 6.9 (Beyond the slides). In the reference implementation a Gaussian's footprint is a disc of radius $3\sqrt{\lambda_{\max}(\boldsymbol{\Sigma}_{2D})}$ around $\boldsymbol{\mu}_{2D}$ (three standard deviations), and the tiles overlapped by that disc receive an instance.

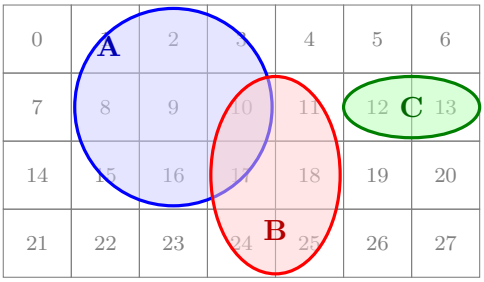


Figure 16: 28 tiles (tile ID $= 7 \cdot \text{row} + \text{col}$) and three Gaussians A, B, C with view depths 2.1, 1.4, 3.0 (reconstructed from slides 35–36). A touches 9 tiles, B 6 tiles and C 2 tiles: 17 instances in total.

6.3.2 Tile Rasterizer 2/3: one radix sort *(slide 36)*

The instances are generated per Gaussian and per overlapped tile, then sorted once with a radix sort on the 64-bit key.

Gaussian	Instances (tile: depth)
A (2.1)	t1, t2, t3, t8, t9, t10, t15, t16, t17
B (1.4)	t10, t11, t17, t18, t24, t25
C (3.0)	t12, t13

After sorting (grouped by tile, front to back inside a tile):
t1:A t2:A t3:A t8:A t9:A t10:**B,A** t11:B t12:C t13:C t15:A t16:A t17:**B,A** t18:B t24:B t25:B

Proposition 6.10 (One global sort gives all per-tile lists)**.** *Sorting instances lexicographically by (tile ID, depth) puts all instances of the same tile in one contiguous block, ordered by increasing depth inside the block.*

Proof. The tile ID occupies the high bits, so it is compared first and instances of different tiles are ordered by tile. Within a tile the low bits decide, i.e. the depth. (For positive IEEE-754 floats, the integer order of the bit pattern equals the numeric order, so the depth bits can be sorted as integers.) □

Intuition

It is like sorting a deck of cards by suit first and rank second with one single pass, instead of dealing the cards into four piles and sorting each pile. The GPU then hands each pile (tile) to one block of threads that already knows whom to draw, front to back. The cost is linear in the number M of instances (here $M = 17$; in a real scene millions).

Remark 6.11 (Popping). Depth is stored *per Gaussian*, not per pixel (slide 36: "depth per Gaussian, not per pixel → popping"). Two overlapping, elongated Gaussians may be at the same depth order at one pixel but swapped at another. With a single global order, a small camera motion can flip the centre depths of two Gaussians, and the image changes abruptly (pops).

6.3.3 Tile Rasterizer 3/3: Blend Each Tile *(slide 37)*

Algorithm 6.12 (Per-pixel blending, runs in parallel for every pixel)**.**

```
    # runs in parallel for every pixel
def render_pixel(pixel, tile):
```

```
T, C = 1.0, 0.0
for g in sorted_gaussians[tile]:          # front to back
    d = pixel - g.mean2d
    a = min(0.99, g.opacity
            * exp(-0.5 * d @ inv(g.cov2d) @ d))
    if a < 1 / 255:
        continue                          # too faint
    if T * (1 - a) < 1e-4:
        break                             # pixel saturated
    C += g.color * a * T
    T *= 1 - a
final_T[pixel] = T                        # kept for backward
return C
```

The four ingredients flagged on the slide: **front to back** order; **early stop** when the pixel is saturated; **skip** $\bar{\alpha} < 1/255$; **clamp** $\bar{\alpha} \leq 0.99$.

Intuition

Front to back lets us stop as soon as the pixel is essentially opaque ($T < 10^{-4}$): everything behind cannot change the colour. Skipping very faint splats ($< 1/255$, below one 8-bit colour level) saves work, and clamping $\bar{\alpha} \leq 0.99$ guarantees $1 - \bar{\alpha} \geq 0.01$, so that the backward pass, which divides by $1 - \bar{\alpha}$, is numerically safe.

6.4 How to model/aggregate appearance? *(slide 38)*

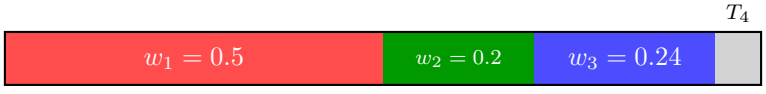
Compute per-Gaussian weights from the opacities of the current and all previous (closer) Gaussians, take the view-dependent colour $\boldsymbol{c}_k$ from the SH coefficients and the ray direction, and aggregate:

$$w_k = \bar{\alpha}_k \prod_{j=1}^{k-1} (1 - \bar{\alpha}_j), \qquad \boldsymbol{c} = \sum_k w_k \boldsymbol{c}_k.$$

Proposition 6.13 (Weights telescope). *With $T_k = \prod_{j<k}(1 - \bar{\alpha}_j)$ we have $w_k = T_k - T_{k+1}$, hence $\sum_{k=1}^{N} w_k = 1 - T_{N+1} \leq 1$.*

Proof. $T_k \bar{\alpha}_k = T_k - T_k(1 - \bar{\alpha}_k) = T_k - T_{k+1}$; summing the telescoping series gives $T_1 - T_{N+1} = 1 - T_{N+1}$. □

Example 6.14. For $\bar{\alpha} = (0.5, 0.4, 0.8)$: $T = (1, 0.5, 0.3, 0.06)$, $w = (0.5, 0.2, 0.24)$, $\sum w = 0.94 = 1 - 0.06$. With colours red, green, blue the pixel is $(0.5, 0.2, 0.24)$, and 6% of the light is left over for whatever is behind (e.g. the background colour).



total light budget = 1: each Gaussian takes its share $\bar{\alpha}_k T_k$ of what is left

Figure 17: Light budget of the example: front Gaussians take their share first; later ones only see the transmittance T_k that remains.

Intuition

Think of a pixel as a budget of 1 unit of light. The nearest Gaussian absorbs the fraction $\bar{\alpha}_1$; the next one absorbs $\bar{\alpha}_2$ *of what remains*, and so on. This is exactly the volume-rendering bookkeeping $T_i\alpha_i$ of the recap, with $\bar{\alpha}_k$ playing the role of α_i, but evaluated only at the primitives instead of at many empty samples.

7 3D Gaussian Splatting: The Full Pipeline *(slides 39–41)*

7.1 The Full Pipeline *(slide 39)*

- **Forward:** project → sort → blend.
- **Backward:** gradients to every Gaussian.
- **Density control:** add and remove Gaussians.

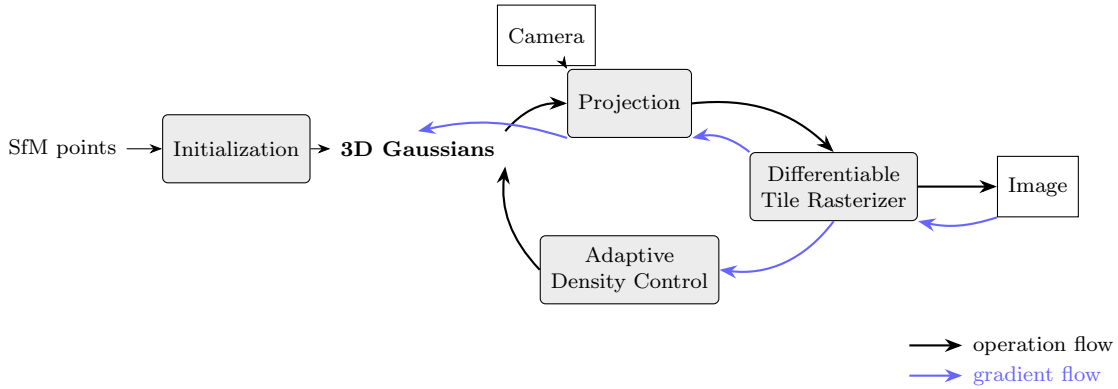


Figure 18: The 3DGS pipeline (Kerbl et al., SIGGRAPH 2023), redrawn from slide 39. Black arrows: operation flow; blue arrows: gradient flow. Gradients from the rasterizer reach the projection (and hence the Gaussian parameters) and also drive the adaptive density control.

7.2 Backward Pass: Gradients Through Blending *(slide 40)*

Let $\boldsymbol{c} = \sum_{i=1}^{N} \boldsymbol{c}_i\, \bar{\alpha}_i\, T_i$ with $T_i = \prod_{j<i}(1-\bar{\alpha}_j)$.

Proposition 7.1 (Gradients of front-to-back blending)**.**

$$\frac{\partial \boldsymbol{c}}{\partial \boldsymbol{c}_i} = \bar{\alpha}_i T_i, \qquad \frac{\partial \boldsymbol{c}}{\partial \bar{\alpha}_i} = T_i \boldsymbol{c}_i - \frac{1}{1-\bar{\alpha}_i} \sum_{j>i} \boldsymbol{c}_j \bar{\alpha}_j T_j.$$

Proof. $\boldsymbol{c}_i$ appears only in term i, with coefficient $\bar{\alpha}_i T_i$. The opacity $\bar{\alpha}_i$ appears (a) in term i as the factor $\bar{\alpha}_i$, giving $T_i \boldsymbol{c}_i$ (because T_i does not depend on $\bar{\alpha}_i$), and (b) inside T_j for every $j > i$ through the factor $(1-\bar{\alpha}_i)$, so $\partial T_j / \partial \bar{\alpha}_i = -T_j/(1-\bar{\alpha}_i)$, contributing $-\sum_{j>i} \boldsymbol{c}_j \bar{\alpha}_j T_j/(1-\bar{\alpha}_i)$. □

Algorithm 7.2 (Backward pass: walk back to front, recover T from the final T)**.**
Since $T_{i+1} = T_i(1-\bar{\alpha}_i)$ we get $T_i = T_{i+1}/(1-\bar{\alpha}_i)$. Only the final transmittance T_{N+1} (`final_T`) must be stored in the forward pass. With upstream gradient $\boldsymbol{g} = \partial \mathcal{L}/\partial \boldsymbol{c}$:

```
T = final_T[pixel];  S = 0                  # S = sum_{j>i} c_j a_j T_j
for i in reversed(range(N)):                # back to front
    T = T / (1 - a[i])                      # now T = T_i
```

```
grad_c[i] = a[i] * T * g
grad_a[i] = g @ (T * c[i] - S / (1 - a[i]))
S += c[i] * a[i] * T
```

Example 7.3. Scalar colours $c = (1, 0.5, 1)$, $\bar{\alpha} = (0.5, 0.4, 0.8)$, so $T = (1, 0.5, 0.3)$. Then $\partial c/\partial \bar{\alpha}_2 = T_2 c_2 - \frac{1}{1-0.4} c_3 \bar{\alpha}_3 T_3 = 0.25 - \frac{0.24}{0.6} = -0.15$. Directly: $c = 0.9 - 0.15\,\bar{\alpha}_2$, whose derivative is -0.15. ✓

Intuition

The two terms in $\partial \boldsymbol{c}/\partial \bar{\alpha}_i$ are a tug of war. Making Gaussian i more opaque (1) adds more of its *own* colour (positive term $T_i \boldsymbol{c}_i$), but (2) hides everything behind it (negative term, the colour that would have come from behind). Whether the net effect is positive depends on whether $\boldsymbol{c}_i$ is brighter or darker (relative to the loss) than the stuff behind. Walking back to front lets us accumulate "the stuff behind" in the running sum S with no per-sample storage; the clamp $\bar{\alpha} \leq 0.99$ makes the division safe.

The upstream gradient $\partial \mathcal{L}/\partial \bar{\alpha}_i$ is then propagated through $\bar{\alpha}_i = \alpha_i \exp(\cdot)$ to α_i, $\boldsymbol{\mu}_{2D}$ and $\boldsymbol{\Sigma}_{2D}$, and through the projection ($\boldsymbol{J}$, $\boldsymbol{W}$) and the factorisation to $\boldsymbol{\mu}, \boldsymbol{s}, \boldsymbol{q}$; the SH coefficients receive $\partial \boldsymbol{c}/\partial \boldsymbol{c}_i$.

7.3 Optimization *(slide 41)*

The loss mixes an ℓ_1 term and a structural term, $\mathcal{L} = (1-\lambda)\mathcal{L}_1 + \lambda \mathcal{L}_{\text{D-SSIM}}$ with $\lambda = 0.2$. Training runs for 30K iterations of Adam, one full image per step, with per-parameter learning rates:

Parameter	Learning rate
position $\boldsymbol{\mu}$	$1.6 \cdot 10^{-4} \rightarrow 1.6 \cdot 10^{-6}$
SH coefficients	$2.5 \cdot 10^{-3}$
opacity	$2.5 \cdot 10^{-2}$
scale	$5 \cdot 10^{-3}$
rotation	10^{-3}

Intuition

Different parameters live on different scales, so one learning rate does not fit all: opacity (a number in $(0, 1)$) can move fast, while positions are in scene units and must settle gently, hence the decay by a factor 100 over training.

8 Initialization, Densification and Ablations *(slides 42–45)*

8.1 Initialization: SfM Points vs Random *(slide 42)*

- One Gaussian per SfM point.
- Isotropic, $\alpha = 0.1$.
- Random initialization gives a worse background.

8.2 Adaptive Density Control *(slide 43)*

Triggered by **large positional gradients**:

[**insert image here**]
taken from **slide 42**

Figure 19: Garden scene: random initialization (top row) vs. SfM initialization (bottom row) and a zoomed crop; the random init is visibly worse in the background. From Kerbl et al., 2023.

- **Small** Gaussian: *clone*.
- **Large** Gaussian: *split*, scale $\div 1.6$.
- **Prune** $\alpha < 0.005$.
- **Reset** α every 3000 iterations.

Algorithm 8.1 (Adaptive density control)**.**
For each Gaussian whose accumulated view-space positional gradient exceeds a threshold: if it is small, clone it (copy, then let optimisation separate the two); if it is large, replace it by two Gaussians with scales divided by 1.6. Remove Gaussians with $\alpha < 0.005$. Every 3000 iterations, reset all opacities to a low value.

Remark 8.2 (Beyond the slides). In Kerbl et al. the positional-gradient threshold is $\tau_{pos} = 2 \cdot 10^{-4}$, densification runs every 100 iterations until about iteration 15K, and the two Gaussians of a split are placed by sampling positions from the original Gaussian. Resetting opacity lets Gaussians that are really needed regain opacity within a few iterations while floaters stay transparent and are pruned.

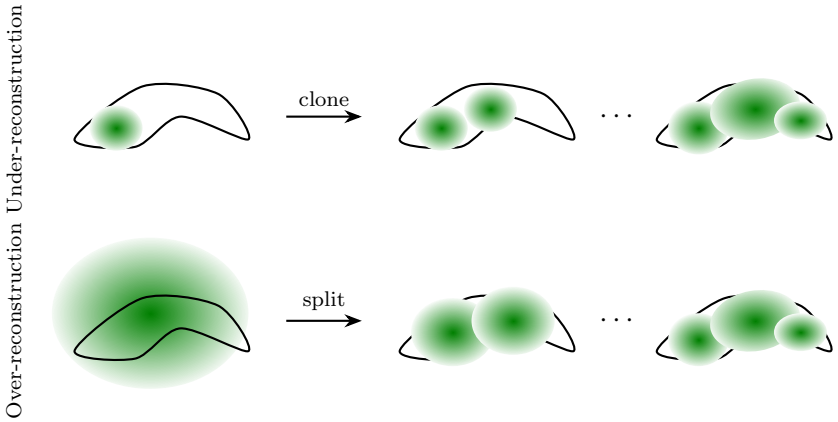


Figure 20: Adaptive density control (redrawn from slide 43): a too small Gaussian that does not cover the shape is cloned; a too large Gaussian that blurs the shape is split into smaller ones. Optimisation then continues.

Intuition

A large positional gradient means "many pixels disagree about where this Gaussian should be": it is trying to be in several places at once. If it is already small, the geometry is simply

under-covered, so we add a copy (clone). If it is big, one blob is trying to explain structure that needs detail, so we replace it by smaller ones (split, $\div 1.6$ in scale). Pruning removes Gaussians that contribute almost nothing.

8.3 Ablation: Densification *(slide 44)*

[**insert image here**]
taken from **slide 44**

Figure 21: Bicycle crop, three versions: **No split** (blurry background), **No clone** (broken spokes, the arrow marks the wheel), **Full**. From Kerbl et al., 2023.

Without splitting, large Gaussians cannot be refined and the background stays blurry; without cloning, thin structures such as spokes are not covered by enough Gaussians.

8.4 Ablation: Anisotropy *(slide 45)*

[**insert image here**]
taken from **slide 45**

Figure 22: Potted plant: ground truth, full model (anisotropic Gaussians) and the isotropic ablation, which yields blobs instead of leaves. From Kerbl et al., 2023.



1 anisotropic Gaussian ≥ 6 isotropic Gaussians

Figure 23: Toy example (not on the slides): a thin leaf or branch needs a single stretched ellipsoid, but a chain of many round blobs if only isotropic Gaussians are allowed, and the result still looks dotted.

Intuition

Natural scenes are full of thin or flat things (leaves, spokes, walls). An anisotropic Gaussian is a pencil or a sheet of paint; an isotropic one is a ball. Using balls to draw lines needs many more balls and leaves visible beads, as seen in the plant ablation.

9 Results: Quality and Speed *(slides 46–48)*

Method	PSNR↑	SSIM↑	LPIPS↓	Train	FPS	Memory
Plenoxels	23.08	0.626	0.463	25m49s	6.8	2.1 GB
Instant-NGP (big)	25.59	0.699	0.331	7m30s	9.4	48 MB
Mip-NeRF360	27.69	0.792	0.237	48 h	0.06	8.6 MB
3DGS, 7K iter.	25.60	0.770	0.279	6m25s	160	523 MB
3DGS, 30K iter.	27.21	0.815	0.214	41m33s	134	734 MB

Kerbl et al., SIGGRAPH 2023, Table 1 (Mip-NeRF360 dataset).

Example 9.1 (Reading the table)**.** Against Mip-NeRF360, 3DGS (30K) loses 0.48 dB of PSNR but has better SSIM and LPIPS, trains $\approx 69\times$ faster (2880 min vs. 41.5 min) and renders $\approx 2200\times$ faster (134 vs. 0.06 FPS). The price is memory: 734 MB vs. 8.6 MB, about $85\times$. After only 7K iterations (6.5 min) 3DGS already matches Instant-NGP's PSNR with an order of magnitude higher frame rate.

Intuition

NeRF-type models store the scene in a small network and pay at render time (many network queries per ray). 3DGS stores the scene explicitly (millions of Gaussians) and pays in memory, but rendering is just "splat and blend", which GPUs do extremely well. It trades memory for speed.

[**insert image here**]
taken from **slide 47**

Figure 24: Interactive 3DGS viewer on the bicycle scene, showing 57.56 FPS (17.37 ms) with VSync on.

[**insert image here**]
taken from **slide 48**

Figure 25: Interactive 3DGS viewer on an indoor playroom scene, showing 89.37 FPS (11.19 ms).

10 Limitations *(slide 49)*

- **Memory:** millions of Gaussians (hundreds of MB per scene).

[**insert image here**]
taken from **slide 49**

Figure 26: Mip-NeRF360 (left) vs. 3DGS (right) on a train scene: 3DGS reconstructs the foreground structures that Mip-NeRF360 smears.

- **Popping:** the per-Gaussian sort (Remark in the Tile Rasterizer 2/3 subsection).
- **Aliasing under zoom** → Mip-Splatting.
- **No surfaces** → SuGaR, 2DGS.

Intuition

Each limitation is a consequence of a design choice. Explicit storage costs memory. A global depth order per Gaussian is cheap but only approximately correct. A fixed-size screen-space filter does not adapt when zooming. And a volumetric blob has no well-defined surface or normal. The next section shows how swapping the primitive addresses the last one.

11 Other Primitives *(slides 50–54)*

11.1 Other Primitives: Fuzzy Metaballs *(slide 50)*

Keselman and Hebert, *Fuzzy Metaballs: Approximate Differentiable Rendering with Algebraic Surfaces*, ECCV 2022 (Oral). The slide shows a teddy bear fitted from a target mask: the estimated mask and depth start as a small cloud at iteration 0 and are optimised towards the target.

[**insert image here**]
taken from **slide 50**

Figure 27: Fuzzy Metaballs optimisation on a teddy bear: input photo, target mask, estimated mask and estimated depth at iteration 0 (epoch 0.0).

11.2 2D Gaussian Splatting: The Primitive *(slide 51)*

Huang et al., SIGGRAPH 2024. A 3D Gaussian intersected by a plane gives different cross-sections for different viewpoints, whereas a 2D Gaussian is a flat disc whose intersection with every viewing ray is well defined and view-consistent.

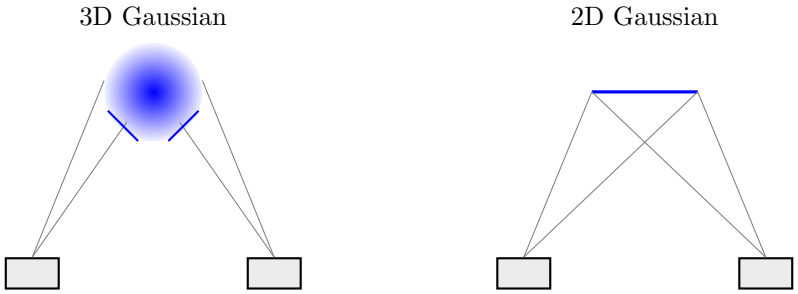


Figure 28: Left: a 3D Gaussian is intersected by planes that differ per viewpoint. Right: a 2D Gaussian is the same planar disc for every camera (schematic, after slide 51).

Definition 11.1 (2D Gaussian splat)**.** A splat has a centre $\boldsymbol{p}_k$, two tangent vectors $\boldsymbol{t}_u, \boldsymbol{t}_v$ with scales s_u, s_v, and lives in its own tangent plane:

$$P(u,v) = \boldsymbol{p}_k + s_u \boldsymbol{t}_u\, u + s_v \boldsymbol{t}_v\, v = \boldsymbol{H}\,(u,v,1,1)^{\mathsf{T}}, \qquad \boldsymbol{H} = \begin{bmatrix} \boldsymbol{RS} & \boldsymbol{p}_k \\ \boldsymbol{0}^{\mathsf{T}} & 1 \end{bmatrix},$$

with $\boldsymbol{R} = [\boldsymbol{t}_u\ \boldsymbol{t}_v\ \boldsymbol{t}_w]$ and $\mathcal{G}(u,v) = \exp\!\big(-\frac{u^2+v^2}{2}\big)$. The splat is **flat**: $\boldsymbol{S} = \mathrm{diag}(s_u, s_v, 0)$; its **normal** is $\boldsymbol{t}_w = \boldsymbol{t}_u \times \boldsymbol{t}_v$.

Intuition

A 2D Gaussian is a fuzzy coin instead of a fuzzy ball. A coin has a definite orientation (its normal), and one ray hits it at one point, so a single hit gives an **exact depth**. A ball only gives a smeared volumetric depth.

11.3 2D Gaussian Splatting: Ray-Splat Intersection *(slide 52)*

1. **A pixel is two planes.** The ray through pixel (x, y) is the intersection of the x-plane $\boldsymbol{h}_x = (-1, 0, 0, x)^{\mathsf{T}}$ and the y-plane $\boldsymbol{h}_y = (0, -1, 0, y)^{\mathsf{T}}$.
2. **Move the planes to the splat frame without an inverse:** $\boldsymbol{h}_u = (\boldsymbol{WH})^{\mathsf{T}}\boldsymbol{h}_x$, $\boldsymbol{h}_v = (\boldsymbol{WH})^{\mathsf{T}}\boldsymbol{h}_y$.
3. **Solve for** (u, v)**:** $\boldsymbol{h}_u \cdot (u, v, 1, 1)^{\mathsf{T}} = \boldsymbol{h}_v \cdot (u, v, 1, 1)^{\mathsf{T}} = 0$.
4. **Edge-on case:** use a screen-space floor, $\hat{\mathcal{G}}(\boldsymbol{x}) = \max\{\mathcal{G}(\boldsymbol{u}(\boldsymbol{x})),\ \mathcal{G}(\frac{\boldsymbol{x}-\boldsymbol{c}}{\sigma})\}$.

Then proceed as in 3DGS: tiles, sort and blend.

Proposition 11.2 (Closed-form ray–splat hit)**.** *Because the third column of* $\boldsymbol{H}$ *is zero* ($s_w = 0$)*, the third component of* $\boldsymbol{h}_u, \boldsymbol{h}_v$ *vanishes and the system in step 3 reads* $h_u^1 u + h_u^2 v + h_u^4 = 0$, $h_v^1 u + h_v^2 v + h_v^4 = 0$*, with solution*

$$u = \frac{h_u^2 h_v^4 - h_u^4 h_v^2}{h_u^1 h_v^2 - h_u^2 h_v^1}, \qquad v = \frac{h_u^4 h_v^1 - h_u^1 h_v^4}{h_u^1 h_v^2 - h_u^2 h_v^1}.$$

Proof. Cramer's rule on the 2×2 system. The denominator vanishes exactly when the splat is seen edge-on (the ray is parallel to the splat's plane), which is the degenerate case handled by the floor in step 4. □

Intuition

Two planes ("everything with this x" and "everything with this y") meet in a line, which is the pixel's ray. Pushing both planes into the splat's own coordinates lets us read off the hit (u, v) by solving two linear equations: no matrix inversion, and the Gaussian weight at the hit is $\mathcal{G}(u, v)$ exactly, rather than a projected approximation. When the coin is seen edge-on it has no area on screen, so we keep a small blob of fixed screen size to avoid losing it (and its gradients).

11.4 From Cameras to LiDAR: Splat-LOAM *(slide 53)*

Giacomini, Di Giammarino, De Rebotti, Grisetti and Oswald, ICCV 2025. The pipeline: a **LiDAR scan → range image**; a **spherical 2DGS rasterizer**; **range-only supervision**; **map + odometry**.

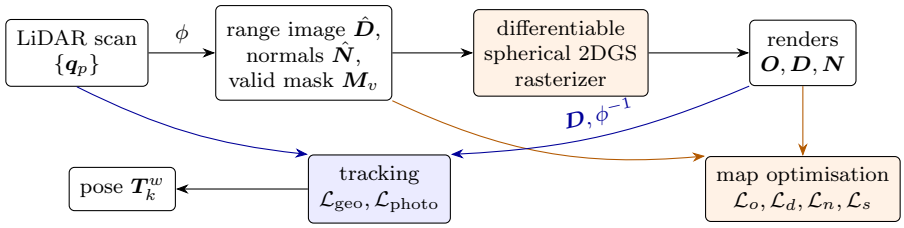


Figure 29: Splat-LOAM, schematically redrawn from slide 53 (block names from the slide; the exact loss definitions are in the paper). The map is optimised from range images, while tracking estimates the sensor pose T_k^w against the rendered depth.

Example 11.3 (Spherical projection to a range image)**.** A LiDAR point $q = (x, y, z)$ has range $r = \|q\|$, azimuth $\theta = \text{atan2}(y, x)$ and elevation $\varphi = \arcsin(z/r)$. A range image stores $D(u, v) = r$ at $u \propto \theta$, $v \propto \varphi$ (the standard spherical camera model, not detailed on the slide).

[**insert image here**]
taken from **slide 53**

Figure 30: Splat-LOAM results panel: the rendered normal map of the reconstructed building (right part of the figure).

11.5 Beyond Gaussians: Other Primitives *(slide 54)*

- **Voxels:** SVRaster (Sun et al., CVPR 2025).
- **Triangles:** Triangle Splatting (Held et al., 2025).
- **Voronoi cells:** Radiant Foam (Govindarajan et al., 2025).



Figure 31: Three alternative primitives on the garden scene: sparse voxels (SVRaster), triangles (Triangle Splatting) and Voronoi cells (Radiant Foam).

Intuition

The recipe of this lecture stays fixed: (1) find the primitives that touch a pixel (project and bin them into tiles), (2) order them, (3) blend front to back. What changes from primitive to primitive is only the *footprint* (ellipse, coin, triangle with soft edge, box, polygonal cell) and how its opacity falls off, so that the whole pipeline remains differentiable.

12 Summary *(slide 55)*

Rasterization + blending for efficient rendering. Aggregating 'soft' primitives along a ray allows differentiable rendering. Projecting 3D Gaussians gives 2D rendering via $\boldsymbol{\mu}_{2D} = \pi(\boldsymbol{\mu}_{3D})$, $\boldsymbol{\Sigma}_{2D} = \boldsymbol{J}\boldsymbol{\Sigma}_{3D}\boldsymbol{J}^{\mathsf{T}}$.

Method	Primitive	How it renders / stays differentiable	Main limitation
Hard mesh ray-casting	triangles	closest hit per ray; (α, β) smooth in vertices	jumps at occlusion and depth order; no gradient to hidden faces
Soft Rasterizer / PyTorch3D	soft triangles	sigmoid mask of pixel distance + K closest, depth-softmax weights; primitive-major loop	blur controlled by σ, γ; only K layers
3D Gaussian Splatting	3D Gaussians	EWA projection $\boldsymbol{J}\boldsymbol{\Sigma}\boldsymbol{J}^{\mathsf{T}}$; one radix sort on (tile, depth); front-to-back α-blending with analytic backward	memory, popping, aliasing, no surface
2D Gaussian Splatting	flat Gaussians (discs)	exact ray–splat intersection in splat frame; exact depth and normal	edge-on views need a screen-space floor
Splat-LOAM	spherical 2DGS	range-image rasterizer, range-only supervision, map + odometry	LiDAR only (no appearance)
Other	voxels, triangles, Voronoi cells	same project–sort–blend skeleton with another footprint	see papers

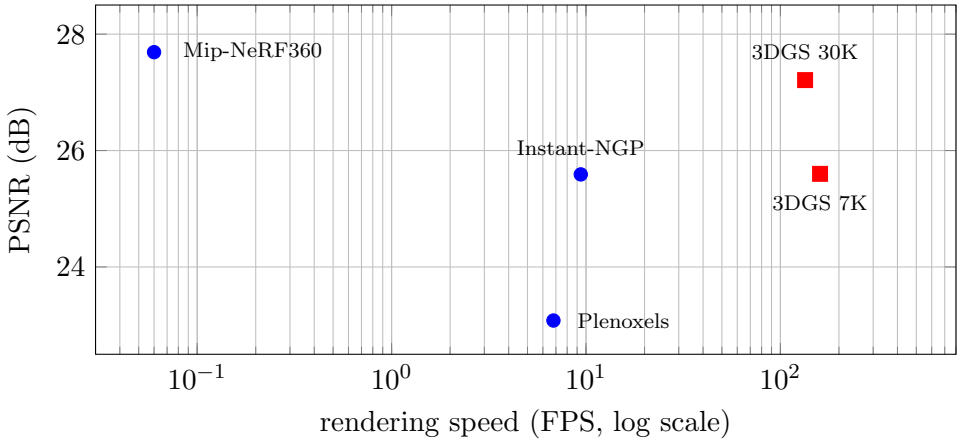


Figure 32: Quality vs. speed on Mip-NeRF360 (data from the results table): the 3DGS points (red) sit towards the top right, i.e. near-best quality at real-time rates.

Key take-aways

1. **Primitive rendering = find influencing primitives + aggregate.** Unlike volume rendering it does not waste samples in empty space.
2. **Hard rendering is not differentiable where it matters:** occlusion and depth ordering create jumps. Soft masks (sigmoid of signed distance) and depth-softmax blending turn the jumps into smooth transitions and give gradients to hidden primitives.
3. **Rasterization swaps the loop order** (primitives first, pixels second), replacing $\Theta(PN)$ tests by $\Theta(N\bar{f} + PK)$; 3DGS does it with tiles and a single radix sort on (tile ID, depth).
4. **3DGS in three formulas:** $\boldsymbol{\Sigma} = \boldsymbol{R}\boldsymbol{S}\boldsymbol{S}^{\mathsf{T}}\boldsymbol{R}^{\mathsf{T}}$ (always PSD), $\boldsymbol{\Sigma}_{2D} = \boldsymbol{J}\boldsymbol{W}\boldsymbol{\Sigma}\boldsymbol{W}^{\mathsf{T}}\boldsymbol{J}^{\mathsf{T}}$ (EWA), and front-to-back blending $\boldsymbol{c} = \sum_k \boldsymbol{c}_k \bar{\alpha}_k T_k$ with a back-to-front analytic gradient that only needs the final transmittance.
5. **Optimisation is more than gradient descent:** adaptive density control (clone, split, prune, opacity reset) and SfM initialisation are essential; limitations (memory, popping, aliasing, no surfaces) motivate Mip-Splatting, 2DGS, SuGaR and other primitives.

Acknowledgments (slide 56). The slides are adapted from *Learning for 3D Vision*, CMU 16-825 (Prof. Shubham Tulsiani); the Gaussian-representation slide is adapted from Vincent Sitzmann. Results and diagrams are from Ravi et al. 2020 (PyTorch3D), Liu et al. ICCV 2019 (Soft Rasterizer), Zwicker et al. 2001 (EWA Volume Splatting), Kerbl et al. SIGGRAPH 2023 (3D Gaussian Splatting), Keselman and Hebert ECCV 2022 (Fuzzy Metaballs), Huang et al. SIGGRAPH 2024 (2D Gaussian Splatting), Giacomini et al. ICCV 2025 (Splat-LOAM), Sun et al. CVPR 2025 (SVRaster), Held et al. 2025 (Triangle Splatting) and Govindarajan et al. 2025 (Radiant Foam).